

Quiz 2: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 2 is Thursday Sep 10, at the end of class. About 12 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Read this first: the scope is narrower than you expect

Two things happened last week. We reached the batching material and did not get through the back half of the deck, and the lecture recording was lost after the Tensors slide. Week 3 will pick up at Tensors and finish Week 2 before we start backpropagation.

So this quiz covers **only the first part of the Week 2 lecture**, plus the activation facts we have now seen three weeks running. It is deliberately shorter and more direct than Quiz 1.

On the quiz:

- What a neuron computes, and what a layer of them computes
- Weight matrix shapes, and counting parameters
- Pre-activation $\mathbf{z}$ versus activation $\mathbf{a}$
- Batching: what shape a batch is, and why the math is unchanged
- Basic activation facts: which ones go in hidden layers, which output activation goes with which task, and why a nonlinearity is not optional

Not on this quiz. These are moving to Quiz 3 with the rest of the Week 2 material, so learn them, but do not spend Wednesday night on them:

- Universal approximation, depth versus width
- Epoch, iteration, batch size, learning rate as vocabulary questions
- Finite-difference gradients and their cost
- Tensors and the TensorFlow Playground demos

Format. Most of the quiz is true/false, multiple choice, and multiple answer. There is one short written section at the end: a parameter count and one neuron written out. Nothing asks you to recall a number from your lab.

Part 1: One neuron

A neuron takes an input vector, weighs each component, adds a bias, and passes the total through a function:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \sigma(z)$$

Two names to keep straight, because the quiz uses them:

- z is the **pre-activation**, the linear part, before σ is applied.
- a is the **activation**, the value after.

Three facts worth having cold:

- **Every neuron has its own bias.** One per unit, never one per input.
- **The input layer computes nothing.** It is the feature vector itself. No weights, no bias, no activation.
- **Weights start random**, not at zero, and training moves them from there.

Part 2: A layer is a matrix multiply

Put several neurons side by side **in the same layer**. They all see the same $\mathbf{x}$, and each has its own weight vector and bias. Stack those weight vectors as the **rows** of one matrix and the whole layer is one line:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

One row of $\mathbf{W}$ per unit, one column per input. This single fact answers most of the shape questions.

Object	Shape
$\mathbf{W}$ for a layer	$(n_{\text{out}} \times n_{\text{in}})$
$\mathbf{b}$ for a layer	(n_{out})
batch input $\mathbf{X}$	$(N \times n_{\text{in}})$, one row per example

Two rules that resolve everything:

1. **Columns of $\mathbf{W}$ must match rows of $\mathbf{x}$** , or the multiply is undefined.
2. **The bias has one entry per unit**, never per input.

One consequence that came up in class: **the architecture is fixed once you choose it**. Adding units to a trained layer changes that layer's $\mathbf{W}$ and every $\mathbf{W}$ downstream, so the trained weights no longer line up.

Also worth knowing: “**feed-forward layer**,” “**linear layer**,” and “**fully connected layer**” all mean this **same thing**. You will hear all three.

Parameter counting

This is the calculation most likely to appear. Per layer:

$$n_{\text{out}} \times n_{\text{in}} + n_{\text{out}}$$

Do it layer by layer and add. **Forgetting the biases is the most common error.** For the 2-3-2-1 network from lecture and Lab 2:

$$\underbrace{(3 \times 2 + 3)}_9 + \underbrace{(2 \times 3 + 2)}_8 + \underbrace{(1 \times 2 + 1)}_3 = 20$$

Practice on a few others until the pattern is automatic. A layer with 5 units and 3 inputs has $5 \times 3 + 5 = 20$ parameters and a 5×3 weight matrix.

Part 3: Batches

Everything above used a single example. In practice you send many at once. $\mathbf{X}$ has shape $N \times D$: **one row per example**, one column per feature.

Because $\mathbf{X}$ stores examples as rows while $\mathbf{W}$ stores units as rows, the batched version carries a transpose:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}$$

Check the shapes: $(N \times D)(D \times H) = (N \times H)$, one row of pre-activations per example.

The batch dimension comes **first** and the layers never touch it. Rows do not interact inside a layer, so pushing 64 examples through at once gives exactly what pushing one through 64 times gives. That is why the same code handles both.

Part 4: Activations

Applied element-wise

σ is applied to each entry separately: $a_{11} = \sigma(z_{11})$, and no other entry of $\mathbf{Z}$ enters that computation. $\mathbf{A}$ always has exactly the same shape as $\mathbf{Z}$.

Hidden layers

Activation	Range	Note
sigmoid	$(0, 1)$	saturates; flat slope far from 0
tanh	$(-1, 1)$	same saturation, but zero-centered
ReLU	$[0, \infty)$	slope 1 for positive z , cheap. The default today.

Softmax is an **output-layer** activation, not a hidden-layer one. MSE is a **loss**, not an activation at all.

The output layer is not a free choice

It is dictated by the problem:

Problem	Output activation	Loss
Scalar regression	none (identity)	MSE or MAE
Binary classification	sigmoid	binary cross-entropy
Multi-class, one label	softmax	categorical cross-entropy

A house price is unbounded, so the output has to be free to take any value; that is why regression gets no output activation.

Why a nonlinearity is not optional

This is the load-bearing idea of the week. Stack linear layers with no activation and they **collapse into a single linear layer**:

$$\mathbf{W}^{(3)} (\mathbf{W}^{(2)} (\mathbf{W}^{(1)} \mathbf{x})) = (\mathbf{W}^{(3)} \mathbf{W}^{(2)} \mathbf{W}^{(1)}) \mathbf{x} = \mathbf{W}_{\text{eff}} \mathbf{x}$$

A composition of linear maps is linear. The depth buys **no expressive power at all**: a 10-layer network with no activations can represent nothing that linear regression cannot. Inserting any nonlinearity breaks the collapse, because it cannot be absorbed into a matrix product.

You saw this directly in Lab 2 Part 3: a 97-parameter stack with no activation drew the identical straight line as a 2-parameter linear layer, and adding tanh was what let the same 97 parameters bend.

Checklist

Before Thursday, confirm you can:

- ☐ Say what z is and what a is, and which comes first
- ☐ State the shape of $\mathbf{W}$ and $\mathbf{b}$ for a layer given its unit count and input count
- ☐ Count parameters layer by layer, biases included, for something like $2 \rightarrow 3 \rightarrow 2 \rightarrow 1$
- ☐ Write one neuron's pre-activation out in full: weights times inputs, plus bias
- ☐ Give the shape of a batch $\mathbf{X}$, and say why the batched forward pass needs a transpose
- ☐ Name the three common hidden-layer activations, and say why softmax and MSE are not among them
- ☐ Name the output activation for regression, binary, and multi-class problems
- ☐ State what stacking linear layers with no activation collapses to, **and why**
- ☐ Explain why the input layer has no parameters
- ☐ Arrive with a calculator and nothing else; no notes are permitted